

LA CANTINA
Test Plan Document
Versione 1.1



Progetto: LA CANTINA	Versione: 1.0
Documento: Test Plan Document	Data: 30/12/2025

Coordinatore del progetto:

Nome	Matricola
GRAZIOSO Fabrizio	0512122950

Partecipanti:

Nome	Matricola
LIGUORI Alessandro	0512119887
CICALESE Gabriele	0512116443
GRAZIOSO Fabrizio	0512122950

Scritto da:	GRAZIOSO Fabrizio
--------------------	-------------------

Revision History

Data	Versione	Descrizione	Autore
22/12/2025	1.0	Scrittura documento	Grazioso, Cicalese, Liguori
30/12/2025	1.1	Correzioni strutturali documento per renderlo compatibile con la nuova versione dell'ODD	Grazioso

Sommario

1.	Introduzione	4
2.	Relationship to other documents	4
3.	System Overview	4
4.	Featured to be tested / not to be tested	5
5.	Pass/Fail Criteria	5
6.	Approach	5
6.1.	Livelli di test	5
6.2.	Tipi di test	6
6.3.	Strategia di Integration testing	6
6.4.	Regressione	7
7.	Suspension and resumption	7
8.	Testing Materials	7
9.	Test Cases	7
9.1.	UserManagement	7
9.2.	Catalog Management	8
9.3.	OrderManagement (Include Carrello)	8
9.4.	Funzionalità amministrative	8
10.	Testing schedule	9

1. Introduzione

Questo documento definisce il Test Plan del sistema LaCantina (SUT – System Under Test) e descrive ambito, approccio, risorse e pianificazione delle attività di test.

L'obiettivo è verificare che il comportamento osservato del sistema sia conforme ai requisiti funzionali e non funzionali descritti nel RAD e che l'implementazione rispetti l'architettura descritta in SDD e ODD.

Il piano copre: Unit Testing, Integration Testing, System Testing, Acceptance Testing e Installation Testing. Per la derivazione dei casi di test funzionali verranno utilizzate principalmente tecniche di Black-Box Testing (equivalence partitioning, boundary value analysis e category partition).

Nota sui sottosistemi: la decomposizione corrente prevede tre sottosistemi principali (UserManagement, CatalogManagement, OrderManagement). La gestione del carrello e degli ordini è inclusa nel sottosistema OrderManagement; le funzionalità amministrative sono esposte attraverso i servizi dei sottosistemi, con controllo accessi basato sui ruoli.

2. Relationship to other documents

Questo piano di test è strettamente correlato alla documentazione di progetto prodotta nelle fasi precedenti:

- **Requirements Analysis Document (RAD):** I test sono progettati per verificare ogni caso d'uso (nel dettaglio UC1 – UC9) ed i requisiti funzionali descritti nel documento stesso
- **System Design Document (SDD):** L'ambiente di test e la configurazione delle varie componenti rispettano l'architettura MVC ed il mapping hardware/software (Tomcat, MySQL) definiti nel documento
- **Object Design Document (ODD):** I test di unità fanno riferimento alle interfacce delle classi definite nel documento (Entity, DAO, Controller).
 - *Naming scheme:* Ogni test case sarà associato ad un caso d'uso tramite la sigla TC_Name_00, in corrispondenza dei vari UC specificati

3. System Overview

LaCantina è una piattaforma web e-commerce per la promozione di vendita di prodotti km0 ad alta qualità. I vari sottosistemi di cui si compone sono:

- **Usermanagement:** Gestione dell'utente e tutte le funzionalità annesse
- **CatalogManagement:** Possibilità di visitare ed interagire con il catalogo prodotti
- **OrderManagement:** Gestione degli acquisti dei clienti e del carrello

Inoltre, gli attori principali sono Utente ed utente con ruolo di Amministratore. L'architettura è la classica architettura MVC con un'organizzazione a vari layer. I componenti principali sono

- **Client web**
- **Web Application Serve**
- **DBMS**

4. **Featured to be tested / not to be tested**

Saranno testate i vari casi d'uso previsti nel **RAD** (UC1 – UC9) ed altre interazioni come

- Consultazione storico ordini e stato ordine
- Filtraggio prodotti per categoria
- Compilazione form di supporto/contatti

NON saranno testate tutte le funzionalità inerenti alle performance strutturali della piattaforma (come gestione di carichi elevati), abilità della piattaforma a rispondere e gestire attacchi informatici base e il pagamento reale tramite gateway esterno.

5. **Pass/Fail Criteria**

Un'esecuzione di test è considerata **PASS** se:

- Tutti i test case critici (UC1–UC9 e flussi alternativi) hanno esito PASS.
- Non sono presenti incident aperti di severità Alta (blocca funzionalità core: autenticazione, acquisto, gestione catalogo/ordini).
- Per i test di unità, è raggiunta almeno la copertura di statement e branch sulle classi core (controller/control e DAO), ove misurabile.
- I requisiti non funzionali minimi dichiarati (es. correttezza gestione stock, coerenza ordine, controllo accessi) risultano soddisfatti.

La campagna di test è considerata **FAIL** se uno o più requisiti core non risultano soddisfatti o se sono presenti failure riproducibili senza workaround in funzionalità critiche.

6. **Approach**

Nel progetto è stata adottata una strategia di test a livelli: unit test per verificare la logica di dominio/servizi e integration test per validare i flussi principali delle Servlet con interazione reale su DAO e database. L'automazione è realizzata con JUnit 5, mentre Mockito viene usato per simulare l'ambiente servlet (request/response/session/dispatcher) e rendere i test ripetibili su un DB di test ripristinato tramite seeding.

6.1. **Livelli di test**

1. Unit testing (White Box)

- Scopo: Verificare la correttezza delle singole classi/metodi (In particolare DAO, Entity e porzioni di logica di controllo)
- Tecniche: statement/branch/loop/patch coverage dove necessario (metodi critici: login/registrazione, aggiornamento stock/prezzo, creazione/modifica ordine).
Il test verrà eseguito tramite JUnit e Mockito, simulando le situazioni previste nei TestFrame dei vari Test Case (doc. TestCases Specification Document).

2. Integration Testing (Black box e Structure testing)

- Scopo: Verificare le interazioni tra sotto sistemi (Tutte le componenti del modello MVC devono poter interagire correttamente tra di loro se richiesto)
- Si utilizzerà lo Structure testing per coprire lati del progetto che non sono testabili tramite altre modalità (coperture delle chiamate tra componenti, percorsi principali nel design)

3. System testing (Black Box)

- Scopo: Verificare l'intero sistema rispetto ai requisiti ed i casi d'uso specificati nella documentazione (RAD, SDD)

4. Acceptance testing (Black Box)

- Scopo: Verificare il corretto funzionamento del sistema
- Nota: Visto che il sistema non prevede un deploy di massa finale, il test della piattaforma verrà "simulato" dal team stesso

5. Installation testing (White Box)

- Scopo: I vari test saranno ripetuti una volta che l'ambiente finale sarà pronto all'uso e completato

6.2. *Tipi di test*

Black box

- Derivazione test: **Use case/Scenari** e dai requisiti del **RAD** (Istanze di UC)
- Tecniche di selezione casi:
 - Equivalence Partitioning e Boundary Value Analysis per gli input (Quantità, prezzo ecc...)
 - Category Partition Method come tecnica per strutturare e ridurre i casi utilizzando vincoli ed annotazioni per generare i test frames

White box

- Applicazione: classi **DAO/Control** dove ha senso misurare la copertura. (Scalabili a livello di unità)
 - UserManagement (Login/registrazione/check email)
 - CatalogManagement (CRUD prodotti lato admin, query ricerca/filtri)
 - OrderManagement (creazione ordine, update righe e update stato ordine)

Category Partition Method

- Applicazione: Per form complessi, verranno definiti dei test Frame combinando categoria/valori ammissibili, riducendo combinazioni ridondanti

6.3. *Strategia di Integration testing*

- Big Bang
- Bottom-up
- Top-down
- Sandwich/Modified Sandwich

6.4. Regression

Dopo ogni creazione di un difetto (fault), si eseguirà un **Regression Testing** sui case interessati.

7. Suspension and resumption

- **Criteri di sospensione:** Le attività di test verranno sospese nel momento in cui verrà individuato un “Bug Bloccante” che impedisce le funzionalità base dell’applicazione
- **Criteri di ripresa:** Le attività di test verranno riprese solamente dopo che suddetto bug verrà corretto. Un test sospeso sarà ritestato dall’inizio per verificare che durante la correzione non sono sorti altri problemi

8. Testing Materials

L’ambiente di test si atterrà alle specifiche di sviluppo definite nel SDD in aggiunta ad alcuni software per il testing:

- **Hardware:** Qualsiasi macchinario o dispositivo in grado di permettere l’esecuzione e simulazione del server e del client
- **Plugins/Software per il test:** JUnit5, Mockito
- **Server Software:** Apache Tomcat 9.0 o superiore
- **Database:** MySQL Server
- **Client software:** Browser web moderni in grado di poter verificare la compatibilità e la responsività
- **Dati di test:** Verrà utilizzato un dataset pre-popolato

9. Test Cases

I test cases sono tracciati sui vari casi d’uso (UC) del RAD. Ulteriori dettagli delle combinazioni ammissibili secondo Category Partition sono riportati nel documento *Test Case Specification*.

9.1. UserManagement

TC_UM_01 – Validazione campo e-mail (UC1/UC2)

(Test Frame)

Categoria	Scelta	Esito atteso
Email	Vuota	Errore: campo obbligatorio, evidenziazione input
Email	Formato non valido (mancanza di '@' o '.')	Errore: formato non valido (UC1.1/UC1.2)
Email	Formato valido (nome@provider.tld)	Accettata
Email	Già presente nel DB (solo in fase di registrazione)	Errore: email già registrata (UC2.1)

TC_UM_02 – Validazione password in registrazione (UC2)

(Test Frame)

Categoria	Scelta	Esito atteso
Password	Vuota	Errore: campo obbligatorio (UC2.1)
Password	Assenza di carattere speciale o/e assenza di carattere numerico	Errore: vincolo non rispettato (UC2.1)
Password	Formato della password valido	Accettata

TC_UM_03 – Login: credenziali corrette/errate e accesso admin (UC1, UC1.2, UC1.3)

- Verificare reindirizzamento alla home utente con credenziali valide
- Verificare messaggio di errore e permanenza in pagina login con credenziali errate
- Verificare reindirizzamento alla dashboard admin con credenziali amministratore

9.2. *Catalog Management*

TC_CM_01 – Visualizzazione catalogo e dettaglio prodotto

- Il catalogo mostra nome, descrizione, prezzo, disponibilità ed informazioni sull'azienda fornitrice; il dettaglio mostra informazioni complete e immagini (se previste).

TC_CM_02 – Filtraggio prodotti per categoria

- Selezionando una categoria, la lista mostra solo i prodotti appartenenti alla categoria; con categoria vuota/"tutte" torna l'elenco completo.

9.3. *OrderManagement (Include Carrello)*

TC_OM_01 – Aggiunta/rimozione prodotti dal carrello

- Aggiunta: incremento quantità e aggiornamento totale parziale.
- Rimozione: decremento quantità e rimozione riga quando quantità=0.

TC_OM_02 – Acquisto diretto (UC3)

- Non autenticato: reindirizzamento a login (UC3.1).
- Checkout con campi non validi: evidenziazione e nessun ordine creato (UC3.2).
- Checkout valido: ordine creato e decremento stock; reindirizzamento a pagina ordini.

TC_OM_03 – Acquisto da carrello (UC4)

- Non autenticato: login e ritorno con carrello preservato (UC4.1).
- Checkout con campi non validi: evidenziazione e nessun ordine creato (UC4.2).

9.4. *Funzionalità amministrative*

TC_ADM_01 – Modifica stock prodotti (UC5/UC5.1)

- Inserimento: valore negativo.
- Inserimento: valore positivo valido
- Rimozione: decremento quantità e rimozione riga quando quantità=0.

TC_ADM_02 – Modifica prezzo prodotti (UC6/UC6.1)

- Inserimento: valore negativo.

- Inserimento: valore positivo valido

TC_ADM_03 – Aggiunta prodotto (UC7/UC7.1)

- Inserimento: campi mancanti o invalidi
- Inserimento: campi validati e completati

TC_ADM_04 – Eliminazione prodotto (UC8)

TC_ADM_05 – Modifica stato ordine (UC9)

10. Testing schedule

La pianificazione delle attività segue le scadenze di progetto

Attività	Responsabile	Scadenza
Stesura Test Plan	Grazioso Fabrizio	
Unit Testing (DAO/Model)	Cicalese Gabriele	
System Testing & Bug Fixing	Liguori Alessandro	
Verifica Finale & Report	Grazioso Fabrizio	
Consegna Documentazione	Tutto il Team	22/12/2025